

Опис архітектури

Архітектура WeatherPulse Lviv побудована за принципом “Python керує процесом і показує, а C# рахує задачі”.

Програма завантажує погодні дані для Львова, погодинно (час, температура, вітер, опади) для дня або 2 днів, які обирає користувач, у форматі JSON. Для завдань 1 і 3 використовується один день (Дата 1): для якого сортуються години за температурою і виводиться ТОП-10 найтепліших годин; у завданні 3 з цього ж файлу відбираються “комфортні години” за порогами (температура близька до цільової, вітер і опади не перевищують обмеження) і показується список та кількість таких годин. Для завдань 2 і 4 використовуються два дні (файли): у завданні 2 для кожного дня окремо рахуються агрегати (min/max/avg температури, max/avg вітру, сума опадів), потім рахується різниця між днями, а також обчислюється ТОП-10 годин з найбільшими відмінностями між днями по годинах; у завданні 4 години двох днів порівнюються по НН:ММ, рахується $|\Delta T|$ і виводиться ТОП-10 найбільших стрибків температури. У GUI для задач на 1 день будується одна крива, а для задач на 2 дні — дві криві на одному графіку (Дата 1 і Дата 2) для обраного показника (температура/вітер/опади).

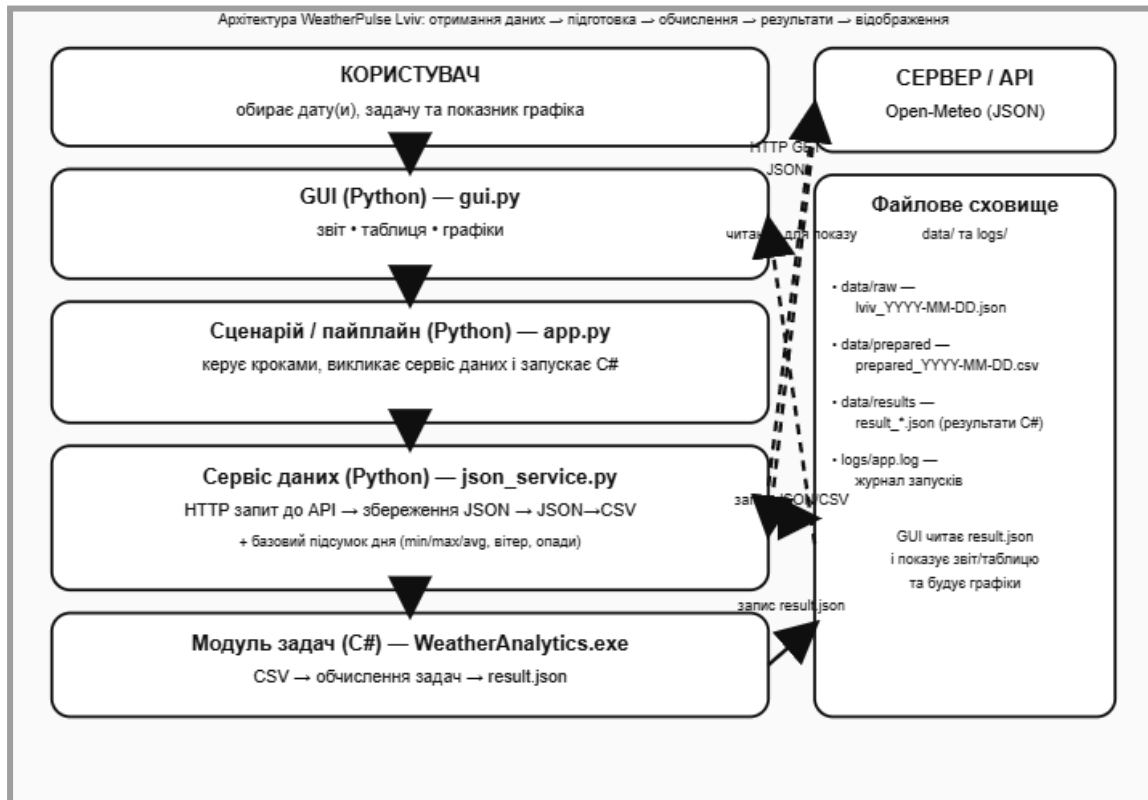
Головна ідея

1. Python (сценарій) отримує погодні JSON-дані з інтернету або бере їх з кешу.
2. Python (сервіс даних) перетворює JSON у CSV єдиного формату (це “контракт” для C#).
3. Python запускає C# модуль як окрему програму (.exe), передає шлях до CSV і отримує JSON-результат.
4. Python GUI перетворює результат у зрозумілий звіт/таблицю та будує графіки.

Ролі файлів

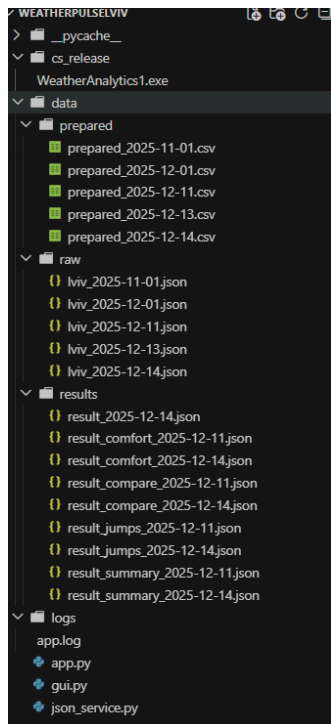
- app.py - головний сценарій (пайплайн): дата/режим → підготовка → запуск C# → повернення результату в GUI
- json_service.py - отримання JSON, кешування, валідація, конвертація JSON→CSV

- gui.py - інтерфейс користувача + таблиці + графіки
- cs_release/WeatherAnalytics.exe - модуль задач (аналіз CSV, розв'язки, формує результат у JSON). Запускається з Python через subprocess.
- в папці data - дані отримані від сервера, підготовлені до обробки, повернені результати після обробки.



Архітектура проекту WeatherPulse Lviv складається з трьох логічних рівнів: рівень взаємодії з користувачем (GUI), рівень сценарію та підготовки даних (Python-пайплайн) і рівень обчислювальних задач (C# модуль). Проект працює з погодними даними для Львова, які отримуються з інтернет-джерела у форматі JSON, кешуються у файловій структурі, перетворюються у стандартизований CSV, а далі аналізуються модулем на C#.

В проекті є папка data/, розділена на підпапки за призначенням: data/raw/ містить сирі JSON-відповіді сервера по кожній даті (наприклад, lviv_YYYY-MM-DD.json), data/prepared/ містить підготовлені CSV-файли для C# (наприклад, prepared_YYYY-MM-DD.csv), а data/results/ містить результати виконання задач C# у форматі JSON (наприклад, result_compare_YYYY-MM-DD.json). Окремо є logs/, де зберігається журнал роботи застосунку.



Python-сценарій (файл `app.py`), який реалізує послідовність кроків виконання. Його можна описати як пайплайн: користувач задає дату(и) та режим, після чого `app.py` координує весь процес: перевіряє наявність сирого JSON у кеші, за потреби робить HTTP-запит до сервера, обробляє можливі винятки (мережеві збої, відсутність даних, неправильна структура відповіді), запускає підготовку CSV і далі викликає модуль задач на C#. У цьому ж файлі зосереджена логіка “яку задачу запускати” і “чи потрібний другий файл”. Для режимів, що потребують порівняння даних двох файлів, сценарій гарантує наявність двох `prepared`-файлів (для дати 1 та дати 2); якщо дата 2 не задана, вона визначається автоматично як “попередній день” відносно дати 1.

Другим ключовим компонентом Python-частини є сервіс даних (`json_service.py`). Він відповідає за отримання даних з API, за запис JSON у `data/raw/`, за валідацію структури відповіді, а також за перетворення даних у підготовлений формат для C#.

Третій компонент — інтерфейс користувача (`gui.py`). Він реалізований на Python (tkinter) і виконує одразу дві функції: керування запуском пайплайна та презентацію результатів. GUI бере введені параметри (дати, вибір задачі/режиму, показник для графіка), викликає функцію пайплайна з `app.py`, отримує у відповідь структуру даних (`meta`, `python_summary`,

cs_result, files) і перетворює це у зручний формат: текстовий звіт, табличне подання результатів задачі (через Treeview), а також графіки (matplotlib).

Рівень задач реалізований як окремий модуль на C#, який компілюється у виконуваний файл WeatherAnalytics.exe (разом із необхідними залежностями .NET) і розміщується в папці cs_release/ у корені проєкту. На вході C# отримує шлях до prepared_day1.csv, шлях до result.json, код задачі та, якщо потрібно, шлях до prepared_day2.csv (для задач порівняння/стрибків). На виході C# створює result.json зі структурою, визначеною самою задачею (наприклад, масив топ-елементів, агрегати, пороги тощо).